```
    newNode2 ← new SsNode(false, children=this.children
[index..])            #6
  return (newNode1,
newNode2)                                                  #7
```

#1 Method `split` is defined on `SsNode`. It returns the two new nodes resulting from the split.
#4 Find the best “split index” for the list of points (leaves) or children (internal nodes).
#5/#6 If this is a leaf, the new nodes resulting from the split will be two leaves, each with part of the points of current leaf. Given `splitIndex`, the first leaf will have all points from the beginning of the list to `splitIndex` (not included), and the other leaf will have the rest of the points list.
#7/#8 If this node is internal, then we create two new internal nodes, each with one of the partitions of the children list.
#9 Return the pair of new `SsNode` created.

This method looks relatively simple, also because most of the leg work is performed by the auxiliary method `findSplitIndex`, described in listing 10.12.

After finding the direction with maximum variance, we sort[11] points or children (depending if a node is a leaf or an internal node) based on their coordinates for that same direction, and then, after getting the list of centroids for node’s entries, we split this list, again along the direction of max variance: we’ll see how to do that in a moment.